

# Experiment 'p\_6\_6\_alternating\_sum\_seq' Results

June 4, 2026

**Experiment outcome:** SUCCESS

**Bad responses:** 0

**Responses containing assume :** 0

**Responses containing expect :** 0

**Resolution attempts:** 1

**Hard fails (resolution):** 0

**Soft fails (resolution):** 0

**Verification attempts:** 1

## Problem Specification

**Problem name:** p\_6\_6\_alternating\_sum\_seq

**Natural language statement:** Compute the alternating sum of all elements in an array. For example, if your method reads the input [1, 4, 9, 16, 9, 7, 4, 9, 11] then it computes  $1 - 4 + 9 - 16 + 9 - 7 + 4 - 9 + 11 = -2$ .

**Method signature:** p\_6\_6\_alternating\_sum\_seq(inputs: seq<int>) returns (alt\_sum: int)

## Ensures

- `alt_sum == sum_array(inputs, 1)`

## Functional Code Given

```
function sum_array(inputs: seq<int>, i: int): int
{
  if |inputs| == 0 then 0
  else inputs[0] * i + sum_array(inputs[1..], i * -1)
}
```

## GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

### Program → GenAI

You are given the following task to perform in Dafny:

Compute the alternating sum of all elements in an array. For example, if your method  
→ reads the input [1, 4, 9, 16, 9, 7, 4, 9, 11] then it computes  $1 - 4 + 9 - 16 + 9 - 7 + 4 - 9 + 11 = -2$ .

The signature should be:

`p_6_6_alternating_sum_seq(inputs: seq<int>) returns (alt_sum: int)`

The method should respect the following contract:

`ensures alt_sum == sum_array(inputs, 1)`

The contract uses the following dafny code:

```
function sum_array(inputs: seq<int>, i: int): int
{
  if |inputs| == 0 then 0
  else inputs[0] * i + sum_array(inputs[1..], i * -1)
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that  
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor  
→ 'expect' anywhere in your code.

### GenAI → Program

**System fingerprint:** null

**ID:** chatcmpl-Dn1ZODDV1pdjTQpYK6xpRfWggu34s

**Model:** gpt-5.5-2026-04-23

**Created at:** 1780576478

**Finish reason:** stop

**Usage:** 550 tokens in, and 6386 tokens out

```
{
  if |inputs| == 0 {
    alt_sum := 0;
  } else if |inputs| == 1 {
    assert inputs[1..] == [];
    alt_sum := inputs[0];
  } else {
    assert 2 <= |inputs|;
    assert 0 < |inputs[1..]|;
  }
}
```

```

    assert (inputs[1..])[0] == inputs[1];
    assert (inputs[1..])[1..] == inputs[2..];
    var rest := p_6_6_alternating_sum_seq(inputs[2..]);
    alt_sum := inputs[0] - inputs[1] + rest;
  }
}

```

## Final Program

```

function sum_array(inputs: seq<int>, i: int): int
{
  if |inputs| == 0 then 0
  else inputs[0] * i + sum_array(inputs[1..], i * -1)
}

method p_6_6_alternating_sum_seq(inputs: seq<int>) returns (alt_sum: int)
  ensures alt_sum == sum_array(inputs, 1)
{
  if |inputs| == 0 {
    alt_sum := 0;
  } else if |inputs| == 1 {
    assert inputs[1..] == [];
    alt_sum := inputs[0];
  } else {
    assert 2 <= |inputs|;
    assert 0 < |inputs[1..]|;
    assert (inputs[1..])[0] == inputs[1];
    assert (inputs[1..])[1..] == inputs[2..];
    var rest := p_6_6_alternating_sum_seq(inputs[2..]);
    alt_sum := inputs[0] - inputs[1] + rest;
  }
}

```

## Total Token Usage

**Input tokens:** 550

**Output tokens:** 6386

**Reasoning tokens:** 6145

**Sum of ‘total tokens’:** 6936

## Experiment Timings

**Overall Experiment** started at 1780576477582, ended at 1780576651446, lasting 173864ms (173.86 seconds)

**Iteration #1** started at 1780576477641, ended at 1780576651446, lasting 173805ms (173.81 seconds)